StreetLight-PK Analytics Dashboard

Civic Intelligence — Feature Design & Free Implementation Guide

From raw reports to predictive area intelligence

Recharts	Supabase	FastAPI	Python	react-calendar-heatmap

Supervised by Dr. Adeel Nissar · PUCIT, University of the Punjab · 2026

TABLE OF CONTENTS

01 Architecture & Data Flow — Three Cluster Model

The analytics dashboard reads directly from the five tables already populated by the AI pipeline — reports, users, report_logs, verification_votes, and report_interactions — and transforms them through three processing clusters into actionable civic intelligence. No new database tables are required for any core metric.

1.1 The Three Processing Clusters

Cluster	Input Tables	What it Produces
Area Profile Cluster	reports (category, severity, status, lat/lng, created_at)	Issue frequency by type, severity distribution, recurrence rate, seasonal patterns per zone
Issue Lifecycle Cluster	reports + report_logs (old_status, new_status, changed_at)	Avg days at each pipeline stage, stall detection, re-report rate, admin response patterns
Citizen Behaviour Cluster	user_profiles (impact_score, fraud_flags) + verification_votes	Badge tier distribution, engagement drop-off, fraud geography, verification rates

1.2 Analysis Methods — No Paid ML Required

Analysis	Method	Schedule
KPI aggregates	SQL COUNT/AVG/GROUP BY on reports + report_logs	On page load (5-min cache)
Re-report rate	SQL self-join: same category within 100m within 14d of RESOLVED	On page load
Pipeline stall	AVG(changed_at - prev_changed_at) per stage via LAG() window function	On page load
30-day forecast	7-week moving average + seasonal index from prior year window	Nightly batch
Area risk score	Weighted formula: density + recurrence + resolution lag + forecast delta	Nightly batch
Citizen health index	Weighted: returning %, avg impact_score, verification %, Paragon count	Nightly batch
Anomaly detection	3-sigma rule on rolling 30-day volume per zone	Nightly batch

02 Admin View — Annotated UI Mockup (Municipal Officer)

The image below is the actual designed admin dashboard for a Municipal Officer viewing the Gulberg III zone — marked HIGH RISK with 43 open reports. This is the view a zone-level admin sees: one zone tab, all KPI cards, both chart panels, three AI insight cards, and the alerts section.

ADMIN VIEW — Gulberg III (HIGH RISK) · Municipal Officer · One zone tab only

2.1 Every UI Element Annotated

Zone / Element	What the Mockup Shows	Data Source
Zone tab strip	Gulberg III (active, white border), Model Town, DHA Phase 5, Johar Town — only own zone is interactive for Municipal Officer	React state + users.assigned_zone
HIGH RISK badge	Red pill next to zone name — Area Risk Score computed nightly	analytics_cache.risk_score
43 open reports sub-label	Count of all non-RESOLVED reports in zone	SQL COUNT WHERE status != RESOLVED AND zone=X
Total Reports KPI	187 with green +23% delta pill	COUNT reports last 30d vs prior 30d
Avg Resolution Time KPI	8.4d with amber +1.2d pill — slower than last month	AVG(updated_at - created_at) for RESOLVED reports
Re-report Rate KPI	31% in red — above 25% threshold — warning triangle icon	Self-join SQL: same category, 100m, 14d after RESOLVED
Community Score KPI	72 with green checkmark — healthy engagement	Weighted formula: returning %, impact score, verification %
Issue Type Breakdown donut	Orange dominant (50% Pothole) with 20% grey Other, 28% teal segment — legend shows Pothole 33%, Garbage 5%, Other 30%	GROUP BY category SQL
Pipeline Lifecycle funnel	Reported 30.1d (widest), narrowing through Under Review 12.5d, Dispatched 8.3d, Disparoted 1.7d, Resolved 3d (green)	LAG() window on report_logs
Next-30d Forecast card	Orange header: +34 reports. Body text: monsoon season prediction	Python moving average + seasonal index
Bottleneck Signal card	Amber IN_PROGRESS tag, 5.1d avg stall diagnostic text	Max stage from pipeline query
Citizen Health Index card	Green checkmark, 68/100, engagement quality description	Weighted composite score
Predictive Alerts section	Dark card with header text and two action buttons	analytics_alerts Supabase table
Generate Full Report button	White-outlined button — triggers PDF export	WeasyPrint + Jinja2 backend
Get Recommendations button	Orange filled button — triggers rule-engine recommendations	Python alert engine JSON

2.2 Design Tokens from the Image

Design Element	Value from Image
Background colour	#0F1626 — deep navy almost black

Design Element	Value from Image
Card background	#1A2540 — slightly lighter navy
Card border	#2A3A5E — subtle blue-navy border, ~0.5px
Primary text	#E8EAF0 — near-white, not pure white
Muted label text	#7A8BAA — mid-blue-grey
Orange accent	#D4611E / #E07830 — burnt orange (matches StreetLight-PK brand)
Red (HIGH RISK / danger)	#C0392B — deep red for badges and critical values
Green (healthy / resolved)	#27AE60 — medium green for positive indicators
Amber (warning / in-progress)	#D4861A — warm amber for bottleneck and warning states
Card corner radius	~4px — subtle rounding, not pill-shaped
Zone tab active state	White border + slightly lighter card background

03 Super Admin View — City-Wide Intelligence Mockup

The Super Admin view uses the exact same dark design language as the admin view but the DATA is fundamentally different. Instead of one zone drilled into, the Super Admin sees the entire city simultaneously: city-wide aggregate KPI totals at the top, all 4 municipal admin area summary cards side by side, city-level aggregate charts, then a cross-area alerts feed ranked by severity.

3.1 Super Admin Layout — Four Distinct Sections

Section	What it Shows	Data Source
A — City-wide KPI strip	4 aggregate KPIs: city total reports (630), city avg resolution (7.2d), city re-report rate (28%), city community score (74) — SUM/AVG across ALL zones.	SQL aggregates with no zone filter (zone=NULL)
B — Municipal Area Breakdown	4 cards side by side, one per municipal admin. Each card shows that admin name, zone name, risk badge, mini-KPIs, top issue, bottleneck, and 30d forecast pill.	Same KPI endpoint called 4x with zone=X each time
C — City-wide charts	Donut: city issue split all zones merged (Pothole 36%, Water Leak 26%, Other 22%, Trash 16%). Pipeline funnel: city avg per stage (22.6d Reported down to 2.9d Resolved). 3 AI mini-cards: city forecast +104, worst bottleneck Johar 11.2d, city health 74/100.	GROUP BY category with no zone filter; LAG() on report_logs all zones
D — Cross-area alerts + buttons	Alerts ranked by severity across all 4 areas. Generate Full Report and Get Recommendations export city-wide data.	analytics_alerts WHERE zone IN (all zones), ORDER BY severity

3.2 Municipal Admin Cards — What Each Card Contains

Card Element	Gulberg III	Model Town	DHA Phase 5	Johar Town
Admin name	Sana Mirza	Kamran Ali	Faiza Noor	Bilal Rauf
Risk badge	HIGH RISK (red)	MED RISK (amber)	LOW RISK (green)	HIGH RISK (red)
Reports (30d)	187	142	67	234
Avg resolution	8.4d (amber)	5.9d (green)	3.2d (green)	11.2d (red)
Re-report rate	31% (red)	18% (green)	9% (green)	44% (red)
Civic score	72	81	91	54
Top issue	Pothole 33%	Trash 43%	Streetlight 45%	Water Leak 42%
Bottleneck	IN_PROGRESS 5.1d	PENDING 1.2d	None detected	PENDING 3.4d
30d forecast	+34 red pill	+18 amber pill	-6 green pill	+58 red pill

3.3 Backend — City-Wide Overview Endpoint

```
# GET /admin/analytics/city-overview (super_admin role only)
async def get_city_overview(sa=Depends(require_super_admin)):
# Section A: city aggregate — no zone filter
city = await supabase.rpc('get_city_aggregate_kpis').execute()
# Section B: each zone summary (4 rows from analytics_cache)
zones = await supabase.table('analytics_cache')
.select('zone,admin_name,total_reports,avg_resolution_days,
re_report_rate,community_score,top_issue,
bottleneck_stage,forecast_30d,risk_score')
```

```
.order('total_reports', desc=True).execute()
# Section C: city charts — same as zone charts but zone param omitted
charts = await get_issue_breakdown(zone=None)
alerts = await supabase.table('analytics_alerts')
.select('*').order('severity').limit(10).execute()
return {'city': city.data, 'zones': zones.data,
'charts': charts, 'alerts': alerts.data}
```

04 Area Zone Tabs — Role-Based Access Control

The tab strip at the top is the primary navigation control. Each tab represents a municipal station zone. The selected tab drives every API call on the page — all queries append ?zone=zone_id. Zero additional backend logic is needed; the zone parameter is a SQL WHERE filter.

4.1 Zone Tab Behaviour by Role

Role	Tabs Visible	Data Scope	Backend Filter
Municipal Officer	1 tab only (own zone)	Own zone data only	Force-assigns zone=users.assigned_zone, override ignored
Super Admin	All 4 zone tabs + aggregate	Full city data	zone=null returns city aggregate; zone=X filters one zone

4.2 React Implementation — Zero Cost

```
const ZONES = [
{ id:'gulberg_iii', label:'Gulberg III', risk:'HIGH', riskColor:'red' },
{ id:'model_town', label:'Model Town', risk:'MED', riskColor:'amber' },
{ id:'dha_phase_5', label:'DHA Phase 5', risk:'LOW', riskColor:'green' },
{ id:'johar_town', label:'Johar Town', risk:'HIGH', riskColor:'red' },
];
const visible = admin.role === 'super_admin'
? ZONES
: ZONES.filter(z => z.id === admin.assigned_zone);
// Tab click sets React state — all API calls re-fire with new zone
const [zone, setZone] = useState(visible[0].id);
useEffect(() => { fetchAllAnalytics(zone); }, [zone]);
```

05 KPI Stat Cards — Four Core Metrics

The four KPI cards are the fastest read in the dashboard. Matching the actual admin view image: each card shows a muted label at top, a large bold value, and a coloured delta pill beneath it. Signal colours follow the same red/amber/green system as the risk badges.

5.1 All Four KPI Cards — Full Specification

KPI Card	Value in Mockup	Delta Pill	Signal	Threshold
Total Reports (30d)	187	+23% (green pill)	Green = stable/falling, Red = spike >20%	No fixed ceiling — direction matters
Avg Resolution Time	8.4d	+1.2d (amber pill)	Amber = slower, Green = faster	City target: 5 days
Re-report Rate	31%	Warning triangle icon (no pill)	Red >25% = fail signal	30% triggers CRITICAL alert
Community Score	72	Green checkmark icon	Green >70 healthy, Amber declining, Red <40	70 = healthy civic engagement

5.2 KPI Endpoint — Single SQL Query for All Four

```
# GET /admin/analytics/kpi?zone=gulberg_iii&days=30
SELECT
COUNT(*) AS total_reports,
COUNT(*) FILTER (WHERE created_at >= NOW()-INTERVAL '30 days')
AS total_30d,
ROUND(AVG(EXTRACT(EPOCH FROM (updated_at-created_at))/86400)
FILTER (WHERE status='RESOLVED'), 1) AS avg_resolution_days,
-- prior window for delta
ROUND(AVG(EXTRACT(EPOCH FROM (updated_at-created_at))/86400)
FILTER (WHERE status='RESOLVED'
AND updated_at BETWEEN NOW()-INTERVAL '60 days'
AND NOW()-INTERVAL '30 days'), 1) AS avg_resolution_prev
FROM reports
WHERE location_city = $zone;
-- Re-report rate computed separately via Python Haversine (see Section 06)
```

5.3 Delta Pill Colour Logic — React

```
// utils/kpiColour.js
export function getDeltaColour(metric, delta) {
const rules = {
total_reports: delta > 0 ? 'red' : 'green',
resolution_time: delta > 0 ? 'amber' : 'green',
```

```
re_report_rate: delta > 25 ? 'red' : delta > 15 ? 'amber' : 'green',
community_score: delta > 0 ? 'green' : delta < -10 ? 'red' : 'amber',
};
return rules[metric] ?? 'gray';
}
```

06 Chart Panels — Donut, Funnel, Trend, Heatmap

Two chart panels are visible in the admin view image: Issue Type Breakdown (donut) and Pipeline Lifecycle Avg Days (funnel). Two more panels — weekly trend line and heatmap calendar — complete the chart layer below. All use Recharts (MIT, free).

6.1 Issue Type Breakdown — Donut Chart (from Image)

The donut in the mockup shows a dominant orange segment (~50%), a dark segment (~28%), and a small teal segment (~20%), plus a legend: Pothole 33%, Garbage 5%, Other 30%. The visual percentages do not perfectly match the legend — the donut shows the colour proportions while the legend shows the actual category breakdown. Clicking a segment cross-filters the map page to that issue type.

Detail	Specification
Library	Recharts PieChart with innerRadius prop creating the donut hole
Colours (matching image)	Orange (#D4611E) = Pothole, Amber (#D4861A) = Garbage, Dark navy (#444B6E) = Other, Teal (#1D9E75) = additional
Click behaviour	Segment click dispatches setIssueFilter(category) to Zustand/Context global state; map component re-renders
Percentage labels	Displayed inside or adjacent to each segment matching the image layout
Legend placement	Right side of donut, custom HTML — Recharts default legend disabled
Data endpoint	GET /admin/analytics/issue-breakdown?zone=X — {category, count, pct}[]
SQL	SELECT category, COUNT(*) as count, ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER(),1) as pct FROM reports WHERE zone=X AND created_at > 30d GROUP BY category

6.2 Pipeline Lifecycle — Funnel Chart (from Image)

The funnel in the image shows five horizontal bars decreasing in width from top (Reported, 30.1 days — widest orange bar) down to Resolved (3 days — short green bar). This visualises where time is lost in the resolution process. The 30.1 days at Reported stage is an immediate signal: reports sit in PENDING for a month.

Stage	Days (Gulberg)	Colour	What long avg means	SQL Source
Reported	30.1d	Orange (widest)	Admin inbox overloaded — triage backlog	PENDING entry timestamp
Under Review	12.5d	Orange	Admin reviewed but not dispatched — decision delay	PENDING to IN_PROGRESS transition

Stage	Days (Gulberg)	Colour	What long avg means	SQL Source
Dispatched	8.3d	Orange	Field team assigned but idle — logistics gap	IN_PROGRESS sub-stage
Disparoted	1.7d	Orange (narrow)	On-site but cannot resolve — skills/resource gap	IN_PROGRESS sub-stage
Resolved	3d	Green	Delay closing tickets after work done	IN_PROGRESS to RESOLVED transition

```
# Pipeline funnel SQL — LAG() window function
SELECT old_status AS stage,
ROUND(AVG(EXTRACT(EPOCH FROM
(changed_at - prev_changed_at))/86400), 1) AS avg_days
FROM (
SELECT old_status, changed_at,
LAG(changed_at) OVER (PARTITION BY report_id ORDER BY changed_at) AS prev_changed_at
FROM report_logs
JOIN reports ON reports.id = report_logs.report_id
WHERE reports.location_city = $zone
) sub
WHERE prev_changed_at IS NOT NULL
GROUP BY old_status ORDER BY MIN(changed_at);
```

6.3 Weekly Trend Line & Heatmap Calendar

Chart	Library	Description	Free Install
Weekly trend line	Recharts LineChart	Daily complaint count + 7-day rolling average overlay. Reveals day-of-week patterns and event-driven surges.	npm install recharts
Heatmap calendar	react-calendar-heatmap	GitHub-style 90-day daily complaint heatmap. Monsoon months visibly glow red across all zones.	npm install react-calendar-heatmap

07 AI Insight Cards — Predictive Intelligence

Three AI insight cards are visible in the bottom row of the admin view: Next-30d Forecast (orange), Bottleneck Signal (amber), and Citizen Health Index (green). None require a paid ML API — all use simple statistical formulas on your existing data.

7.1 Next-30d Forecast Card (from Image)

The image shows: orange header with '+34 reports' tag, large bold '+34 reports' value, and description text referencing monsoon season. The card has an orange left border accent. Computed using a 3-factor model: moving average baseline x seasonal index + trajectory.

Factor	Formula	Gulberg III Example
Moving average baseline	Mean of last 7 weekly report counts x 4	Avg 28/week = 112/month
Seasonal index	Historical this-month avg / annual avg	Aug = 1.30x due to monsoon
Trajectory	Linear slope over last 4 weeks x 4 (30d)	Rising +8% = +9 reports
Final forecast	(baseline x seasonal) + trajectory	(112 x 1.30) + 9 = +34 above current

7.2 Bottleneck Signal Card (from Image)

The image shows: amber 'IN_PROGRESS' tag inline with the label, large '5.1d avg stall' value, and description text. The card identifies which pipeline stage has the longest stall — computed entirely from the report_logs table, zero extra APIs.

```
def detect_bottleneck(stage_avgs: dict) -> dict:
# stage_avgs = {'PENDING': 1.8, 'IN_PROGRESS': 5.1, 'RESOLVED': 1.5}
worst = max(stage_avgs, key=stage_avgs.get)
messages = {
'PENDING': 'Reports not being triaged. Admin staffing issue.',
'IN_PROGRESS': 'Reports linger after assignment. Field team delay suspected.',
'RESOLVED': 'Work done but tickets not being closed promptly.',
}
return {'stage': worst, 'avg_days': stage_avgs[worst],
'message': messages.get(worst, '')}
```

7.3 Citizen Health Index Card (from Image)

The image shows: green checkmark icon, 68/100 in large green text, and description 'Engagement quality score in endvelopement.' (placeholder text in mockup). The real description shows: returning reporter %, avg impact score, Paragon citizen count.

Signal	Weight	Source Column	Formula
Returning reporters	30%	reports + users	% reporters who submitted again in last 30d
Avg impact score	25%	user_profiles.impact_score	AVG(impact_score) normalised to 100
Verification participation	25%	verification_requests + votes	% reports receiving at least 1 vote
Paragon-tier count	20%	impact_score > 800	Count Paragon users in zone / total zone reporters

7.4 Area Risk Score Badge (HIGH RISK pill in Zone Tab)

Input	Weight	HIGH threshold
Complaint density (reports per km2 per day)	30%	Above zone 90th percentile
Re-report rate	25%	Above 30%
Avg resolution time	20%	Above 2x city average (10d+)
30-day forecast delta	15%	Positive and >20% above baseline
Community score	10%	Below 40 (disengaged)

Composite Score	Badge	Display
70 – 100	HIGH RISK	Red pill in zone tab and card header — as seen in Gulberg III
40 – 69	MODERATE	Amber pill
0 – 39	STABLE	Green pill — as seen in DHA Phase 5

08 Predictive Alerts & Recommendations Feed

The alerts section is visible at the bottom of the admin view image as a dark card with the header 'Predictive Alerts & Recommendations'. The full scrollable feed lists auto-generated intelligence messages. Rules run as Python on page load (light) or nightly cron (heavy) and are cached in an analytics_alerts Supabase table.

8.1 All Five Alert Types

Type	Dot colour	Example Message	Trigger Condition
Area Warning	Red	Re-report rate 45% — field verification recommended	re_report_rate > 30%
Resource Suggestion	Blue	Assign 2 extra officers to Gulberg III this week	pending_count > 50 AND forecast spike
Seasonal Prediction	Amber	Pothole reports spike historically in Feb-March	Date within 3 weeks of historical seasonal peak
Department Nudge	Orange	Sanitation avg resolution 12.4d — target is 5d	Any category avg resolution > 2x target (5d)
Anomaly Alert	Red (bold)	DHA Phase 5 volume spiked 3x today	Single-day count > 3 sigma above 30d rolling mean

8.2 Alert Rule Engine — Python Implementation

```
# backend/analytics/alert_engine.py
def run_alert_rules(zone: str, metrics: dict) -> list[dict]:
alerts = []
if metrics['re_report_rate'] > 30:
alerts.append({'type':'area_warning','severity':'high',
'message':f'Re-report rate {metrics["re_report_rate"]}% — field verification recommended',
'meta':'Re-report rate threshold exceeded'})
if metrics['pending_count'] > 50 and metrics['forecast_delta'] > 0:
n = max(1, metrics['pending_count']//25)
alerts.append({'type':'resource','severity':'medium',
'message':f'Assign {n} extra officers to {zone} this week',
'meta':f'Pending: {metrics["pending_count"]} + rising forecast'})
mean = metrics['rolling_30d_mean']; std = metrics['rolling_30d_std']
if metrics['today_count'] > mean + 3*std:
alerts.append({'type':'anomaly','severity':'critical',
'message':f'{zone} spiked {round(metrics["today_count"]/mean,1)}x today',
'meta':f'Today: {metrics["today_count"]} vs avg {round(mean)}'})
return sorted(alerts, key=lambda a:
{'critical':0,'high':1,'medium':2}.get(a['severity'],3))
```

8.3 analytics_alerts Table Schema

```
CREATE TABLE analytics_alerts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
zone TEXT NOT NULL,
type TEXT NOT NULL, -- area_warning|resource|seasonal|dept_nudge|anomaly
severity TEXT NOT NULL, -- critical|high|medium|low
message TEXT NOT NULL,
meta TEXT, -- diagnostic context shown below message
generated_at TIMESTAMPTZ DEFAULT now(),
is_read BOOLEAN DEFAULT FALSE
);
CREATE INDEX alerts_zone_sev ON analytics_alerts(zone, severity, generated_at DESC);
```

09 Citizen Health & Fraud Geography

Four panels in the lower analytics section surface citizen behaviour and data integrity signals. These turn the impact_score and fraud_flags data from the Engine Implementation Docs into infrastructure intelligence.

9.1 Badge Tier Distribution

Horizontal bar chart showing how many citizens in the zone fall into each badge tier. More StreetLight Paragon users = higher average vote weight in Engine C = more reliable community verification.

Badge Tier	Impact Score Range	High count means
Rookie (Street Scout / Road Ranger / Eagle Eye)	0 – 200	Low verification weight — zone has mainly new reporters
Community Hero (Civic Warrior / Street Sentinel / Problem Buster)	201 – 500	Active engaged core — healthy verification quality
Master Reformer (Urban Legend / Infrastructure Icon / Trust Titan)	501 – 800	Dedicated civic community — high vote weight
StreetLight Paragon (City Architect / Beacon / Grand Overseer)	801 – 1000	Elite reporters — votes carry up to 11x weight in Engine C

9.2 Fraud Geography Panel

Fraud Type	Engine B Check	Data Source	Dashboard Display
GPS Spoofing	Check 1 — Impossible Travel	reports WHERE flagged=true, lat/lng	Red dot cluster on mini-map
Duplicate Submissions	Check 2 — Duplicate Detection	reports.duplicate_of_id IS NOT NULL	Orange markers, grouped by area
Spam Bursts	Check 3 — Rate Detection	reports.is_flagged_for_spam = true	Amber zone overlay on high-spam sub-areas

No extra map library

The fraud geography mini-map reuses the Leaflet component from the map page — pass fraudOnly:true as a filter prop. Reports flagged by Engine B are already in the reports table. Zero extra infrastructure.

9.3 Engagement Drop-off & Verification Participation

Metric	Signal	Action Threshold
Returning reporter %	3 consecutive months of decline = citizens losing trust	Below 40% = re-engagement campaign needed
Verification participation %	Low rate = too few nearby verifiers OR app UX issue	Below 20% = verification dead zone
Avg days to first vote	High = Engine C timeout risk	Above 24h = push notification urgency increase
Timeout vs completed ratio	High timeout = Engine C not reaching 3-vote minimum	Above 50% = zone needs FCM campaign

10 Action Buttons & Implementation Roadmap — All Free

Two action buttons are visible in the admin view image at bottom-right: 'Generate Full Report' (white-outlined) and 'Get Recommendations' (orange filled). A third button — Download Raw Data — is part of the full spec. All three are implementable at zero cost.

10.1 Action Buttons — Full Spec

Button	Design (from image)	What it does	Free Implementation
Generate Full Report	White-outlined dark button	Exports all KPIs, charts, AI cards as PDF	FastAPI + WeasyPrint + Jinja2 — pip install weasyprint
Get Recommendations	Orange filled button (brand colour)	Structured action plan: dept nudges, field team deployment	Python rule engine reading analytics_cache — no LLM needed
Download Raw Data	Not in image — add below buttons	CSV of all reports for current zone + filter state	FastAPI StreamingResponse + csv.DictWriter — built-in

10.2 PDF Report Generation — WeasyPrint

```
# GET /admin/analytics/report/pdf?zone=gulberg_iii
from weasyprint import HTML
from jinja2 import Environment, FileSystemLoader
from fastapi.responses import StreamingResponse
import io
@router.get('/admin/analytics/report/pdf')
async def generate_pdf(zone: str, admin=Depends(get_admin)):
metrics = await gather_all_metrics(zone)
html = jinja_env.get_template('report.html').render(**metrics)
pdf = HTML(string=html).write_pdf()
return StreamingResponse(io.BytesIO(pdf),
media_type='application/pdf',
headers={'Content-Disposition':f'attachment; filename={zone}_report.pdf'})
```

10.3 Prioritised Implementation Roadmap

Priority	Feature	Frontend	Backend	Free Tool
P1	KPI stat cards	API call + delta colour logic	GET /analytics/kpi — 4 SQL aggregates	Supabase SQL
P1	Issue type donut	Recharts PieChart, cross-page filter	GROUP BY category query	Recharts MIT

Priority	Feature	Frontend	Backend	Free Tool
P1	Pipeline funnel	Recharts BarChart horizontal	LAG() window on report_logs	Recharts MIT
P2	Bottleneck card	Card reads bottleneck.stage from API	Max stall stage from pipeline query	FastAPI Python
P2	30-day forecast card	Card shows delta + reason string	Moving average Python, nightly cron	Python stdlib
P2	Citizen health index	Score card 0-100 with colour ring	Weighted formula, user_profiles + logs	Supabase SQL
P3	Predictive alerts feed	Scrollable list, coloured dot, dismiss	Rule engine writes analytics_alerts	Python + Supabase
P3	Heatmap calendar	react-calendar-heatmap, 90-day window	Daily count query, cached	react-calendar-heatmap MIT
P3	Badge tier distribution	Recharts BarChart, 4 tiers	CASE WHEN impact_score GROUP BY tier	Recharts MIT
P3	Fraud geography panel	Reuse Leaflet, fraudOnly=true prop	GET /analytics/fraud-map?zone=X	Leaflet (installed)
P3	City comparison panel	Super Admin only conditional render	All zones aggregate query	React conditional
P4	PDF report export	Download button, streaming response	Jinja2 HTML + WeasyPrint PDF	WeasyPrint MIT
P4	Recommendations engine	Structured action card list	Python rule engine, analytics_cache	Python
P4	CSV export	Download button	FastAPI StreamingResponse csv	FastAPI built-in

10.4 Total Cost Summary

Service	Usage	Monthly Cost
Supabase PostgreSQL	All queries, analytics_alerts, analytics_cache tables	Free (500 MB)
FastAPI + Uvicorn	All analytics endpoints	Free / MIT
Railway or Render	Backend hosting	Free tier
Recharts	All chart panels	Free / MIT
react-calendar-heatmap	90-day heatmap	Free / MIT

Service	Usage	Monthly Cost
WeasyPrint	PDF report generation	Free / MIT
Vercel	Frontend hosting	Free tier
TOTAL		$0 / month

StreetLight-PK — Analytics Dashboard Feature Design Document

Supervised by Dr. Adeel Nissar · PUCIT, University of the Punjab · FYP 2026